

CS224 Object Oriented Programming and Design

Methodologies Quiz Question (C++)

Lab Quiz – Mako Reactor Simulation

Scenario

In *Final Fantasy VII*, Mako Reactors operated by the Shinra Electric Company extract the planet's life force to generate electricity for the city of Midgar. As members of AVALANCHE, you prepare to sabotage these reactors. To do so, you must mathematically model their behavior inside `Mako.cpp` using object-oriented programming principles.

Your task is to design a hierarchy of classes that represent different types of mathematical equations.

Part A: Abstract Class Equation

Define an abstract class `Equation` with the following attributes and methods:

Attributes

- **Name:** a `std::string` storing the equation's name.

Methods

- **Constructor:** Takes a `std::string` parameter to initialize `Name`.
- **Pure Virtual Functions:**
 - `double Evaluate(double x) const`: returns the value of the function at x .
 - `double Derivative(double x) const`: returns the derivative of the function at x .
- **Getters:**
 - `string getName() const` : returns the value of the name attribute

Part B: Derived Class Polynomial

Inherits from `Equation`.

Attributes

- `n` : An Integer which represents the number of terms in the polynomial
- **Terms:** a 2D dynamic array (`double**`). Each inner array stores:
 - * Index 0: coefficient
 - * Index 1: power of x

Example representation:

$$[[2, 3][5, 2][2, 4]] \Rightarrow 2x^3 + 5x^2 + 2x^4$$

Methods

- **Constructor:** Initializes the **Terms** attribute using the provided 2D array.
- **Evaluate(double x):** Computes the polynomial value.
Example for $2x^3 + 5x^2 + 2x^4$ at $x = 7$:

$$2(7^3) + 5(7^2) + 2(7^4) = 686 + 245 + 4802 = 5733$$

- **Derivative(double x):** For each term (c, p) , compute:

$$(c \cdot p)x^{p-1}$$

Example for $2x^3 + 5x^2 + 2x^4$:

$$(2 \cdot 3)x^2 + (5 \cdot 2)x + (2 \cdot 4)x^3$$

At $x = 7$:

$$6(7^2) + 10(7) + 8(7^3) = 294 + 70 + 2744 = 3108$$

Part C: Derived Class Exponent

Also Inherits from Equation

Attributes

- **Base:** a double representing the base of the exponential expression.
For example, $\text{Base} = 3$ corresponds to the function 3^x .

Methods

- **Constructor:** initializes **Base**.
- **Evaluate(double x):** returns Base^x .
Example:

$$3^5 = 243$$

- **Derivative(double x):**

$$\ln(\text{Base}) \cdot \text{Base}^x$$

Example:

$$\ln(3) \cdot 3^4 \approx 88.98$$

Part D: Derived Class Logarithmic

Also Inherits from Equation

Attributes

- **Base**: an integer equal to either 2 or 10.

Methods

- **Constructor**: initializes the base (only 2 or 10 allowed).
- **Evaluate(double x)**: computes $\log_{\text{Base}}(x)$.
- **Derivative(double x)**:

$$\text{If Base} = 2: \frac{1}{x \ln(2)} \quad \text{If Base} = 10: \frac{1}{x \ln(10)}$$

Part E: main() Function Requirements

Your `main()` function must:

- Instantiate the following objects:
 - * **Polynomial** “Reactor 1” with terms $[[2, 3] [4, 4] [5, 2]]$
 - * **Exponential** “Reactor 2” with base 3
 - * **Logarithmic** “Reactor 3” with base 10
 - * **Logarithmic** “Reactor 4” with base 2
- Store them in a single array of .
- Ask the user to enter a value for x (test value: $x = 2$).
- Output:
 - * The name of the function with the greatest value at x
 - * The name of the function with the greatest derivative at x
- Ensure proper memory management.

Helpful <cmath> Functions

<code>std::log(x)</code>	Natural logarithm ($\ln x$)
<code>std::log10(x)</code>	Base-10 logarithm
<code>std::log2(x)</code>	Base-2 logarithm

Grading Rubric

Abstract Class Equation	5 Marks
Polynomial Class	10 Marks
Exponent Class	5 Marks
Logarithmic Class	5 Marks
Main and Memory Management	10 + 5 Marks